

Foundations of Prompt Engineering & API Access

Becoming a Professional AI Agent Engineer



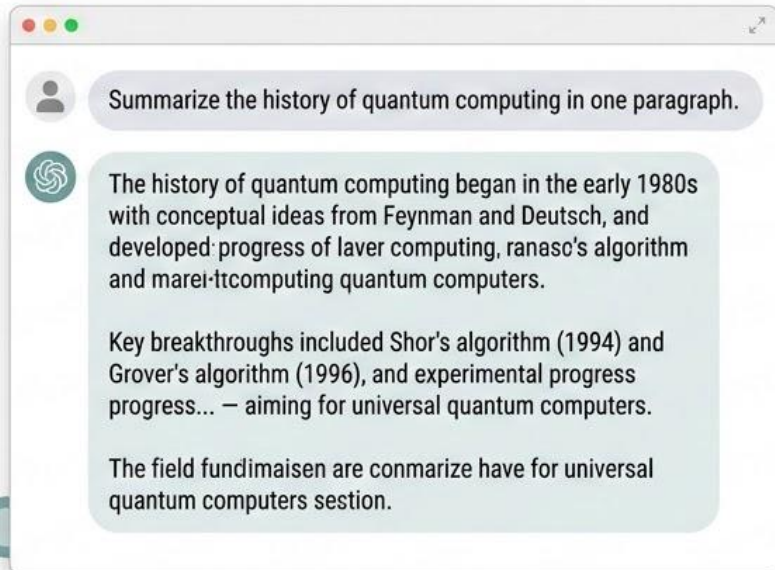
Session Agenda

- 1 UI vs API - The Hinge Moment
- 2 Calling LLMs via API
- 3 Prompt Anatomy
- 4 Core Prompt Engineering Techniques
- 5 Live Exercise & Best Practices

The Hinge Moment – UI vs API

Same Prompt, Same Model, Very Different Results.

Chat UI



Summarize the history of quantum computing in one paragraph.

The history of quantum computing began in the early 1980s with conceptual ideas from Feynman and Deutsch, and developed progress of laser computing, Shor's algorithm and experimental progress quantum computers.

Key breakthroughs included Shor's algorithm (1994) and Grover's algorithm (1996), and experimental progress... — aiming for universal quantum computers.

The field fundamentally are summarized have for universal quantum computers section.

API Call

```
POST https://api.llm.com/v1/chat/completions
{
  "model": "gpt-4-turbo",
  "messages": [
    {
      "role": "user",
      "content": "Summarize quantum computing history in one paragraph."
    }
  ],
  "temperature": 0.5
}

{
  "choices": [
    {
      "message": {
        "role": "assistant",
        "content": "The field originated in the early 1980s, with key theoretical breakthroughs like Shor's algorithm and experimental progress and progress in: aiming in universal quantum computers, aiming for universal quantum computers."
      }
    }
  ]
}
```

What Actually Happens in Chat Interfaces



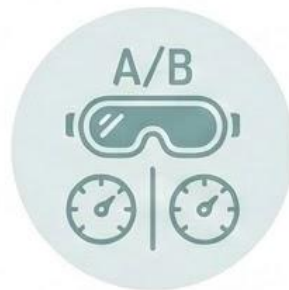
Hidden System Prompts

Invisible instructions guiding the model.



Silent Tool Calling

Automatically accessing external tools.



Different Default Settings

Optimized parameters vs. user controls.



Post-processing

Refining and formatting the final output.

Why Production & Agents Use API

Full Control

Define precise parameters, models, and tool access.

Consistency

Ensure predictable and repeatable responses across applications.

Cost Management

Optimize token usage and monitor spend with granular controls.

Structured Output

Generate structured JSON other formats for integration.

Calling LLMs via API

From Chat to Code

```
import os
from openai import OpenAI

# Configure your API key
client = OpenAI(
    api_key=os.environ.get('OPENAI_API_KEY'),
)

# Basic Chat Completion call
chat_completion = client.chat.completions.create(
    messages=[
        {
            'role': 'user',
            'content': 'Explain prompt engineering simply.',
        }
    ],
    model='gpt-3.5-turbo',
)

print(chat_completion.choices[0].message.content)
```

The Three Core Message Roles

System

Defines the assistant's context and constraints. Set the overall persona and behavior.

```
{  
  'role': 'system',  
  'content': 'You are a helpful and concise study guide assistant.'  
}
```

User

The person or application initiating the conversation. Provides instructions, questions, or content.

```
{  
  'role': 'user',  
  'content': 'Explain prompt roles to me.'  
}
```

Assistant

The AI model's response based on the combined system and user inputs. The completion output.

```
{  
  'role': 'assistant',  
  'content': 'The three roles are System, User, and Assistant. The system role provides context...'  
}
```

Assistant Prefill – Powerful Technique

Before (Standard Prompt)

```
messages=[
    {'role': 'system', 'content': 'You are a
concise study assistant.'},
    {'role': 'user', 'content': 'Outline the
three branches of the US government.'}
]
```

```
chat_completion = client.chat.completions.c
create(
    model='gpt-3.5-turbo',
    messages=messages
)
```



Output starts from scratch.

After (Assistant Prefill)

```
messages=[
    {'role': 'system', 'content': 'You are a
concise study assistant.'},
    {'role': 'user', 'content': 'Outline the
three branches of the US government.'},
    # Prefilling the assistant's first few words
    {'role': 'assistant', 'content': 'Sure! The
three branches are 1.'}
]
```

```
chat_completion = client.chat.
model='gpt-3.5-turbo',
messages=messages
)
```

Injected assistant
prefill forces
completion structure



Output starts precisely after prefill.

Structured Outputs – Must for Agents.

JSON Mode



- Guarantees valid JSON output.
- Simplifies response parsing.
- No external schema definition.

Tool Calling



- Dynamically invokes functions.
- Extracts structured arguments.
- Best for complex, multi-step agent interactions.

XML Tagging



- Clearly defined hierarchical structure.
- Suitable for complex nested data.
- Human-readable format.

Prompt Anatomy – System + User + Assistant



✓ Combined, this defines the full input for the API call.

Positive Framing Beats Negative Rules.

Bad Prompt (Negative)



DON'T return JSON, DON'T use Markdown. Do NOT provide an introduction. Do NOT be conversational.

Good Prompt (Positive)



Provide the response as a valid, compact JSON object. Focus only on the core answer, being concise and direct.

Markdown vs XML Structure

Markdown (Simplicity)

Heading 1

Heading 2

- Point 1
- Point 2

XML (Verbosity)

```
<data>
  <h1 class="main">Heading 1</h1>
  <h2 class="sub">Heading 2</h2>
  <ul>
    <li>Point 1</li>
    <li>Point 2</li>
  </ul>
</data>
```


Zero-Shot vs Few-Shot Prompting.

Comparison Attribute	Zero-Shot Prompting	Few-Shot Prompting
Core Concept	The LLM performs a task based only on the prompt and its pre-training, with zero specific examples.	The LLM learns to perform a task by being provided with a 'few' input/output examples as on as context.
Context Provided	None. Minimal guidance is provided.	Specific examples demonstrating the desired output format or task logic.
Prompt Example (Text/Layout)	Text: 'Translate the following to French: "Where is the library?"' Output: [Direct translation]	Input: "I love this!" -> Output: POSITIVE Input: "This is terrible." -> Output: NEGATIVE Input: "It was okay." -> Output: ??? Output: [Predicted Sentiment Label]
Model Behavior	Relies on broad internal knowledge, sometimes resulting in a less accurate or less formatted output for niche tasks.	Adapts to the provided context, often showing dramatic performance improvements on complex or novel tasks.

COSTAR – Your Go-To Framework.



CONTEXT

Provide the relevant background, situation, or problem details.



OBJECTIVE

Define the end goal or target action required.



TONE

Describe the voice, mood, and feel of the communication.



STYLE

Specify the desired format, structure, or creative direction.



AUDIENCE

Define the intended reader or viewer (e.g., target profile, demographics).



RESPONSE

Set expectations for the output (e.g., length, constraints, preferred examples).



COSTAR Framework – Detailed Breakdown.



CONTEXT

We are launching a new, high-end, IoT-enabled smart coffee maker called the BrewMaster Pro. The market is competitive, focusing on quality and convenience. 🔍



OBJECTIVE

Generate a detailed, persuasive, and accurate product description to increase product interest, convey unique value propositions, and drive pre-orders. 🎯



TONE

Sophisticated yet accessible, innovative, and trustworthy, matching a modern tech lifestyle brand. Avoid heavy technical jargon. 📢



STYLE

Feature-benefit storytelling, starting with an emotional hook. Structure with an opening paragraph, detailed feature breakdown using bullet points, and a summary. 🎨



AUDIENCE

Tech-savvy coffee enthusiasts, busy professionals who appreciate quality and convenience, and early adopters of smart home technology. 👥

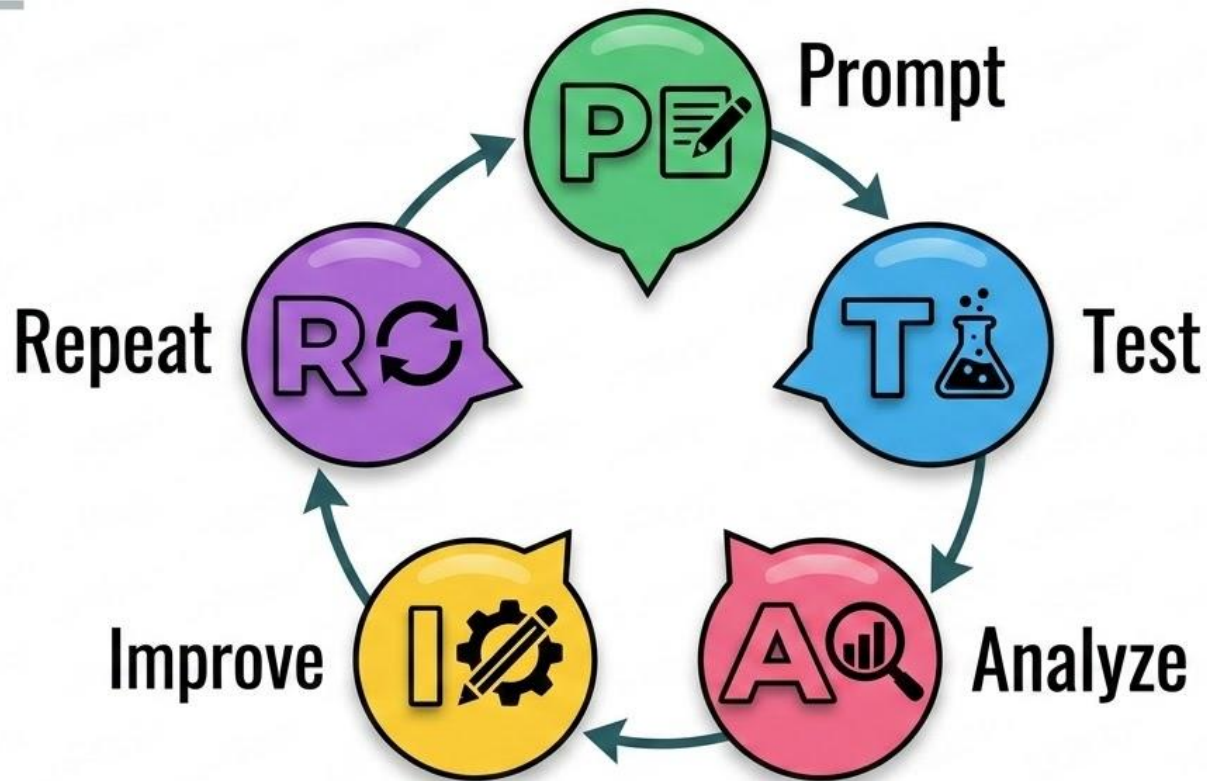


RESPONSE

The description must be 200-250 words in English. Include dimensions, key brewing specifications, and wireless connectivity features. Must follow SEO best practices. 📝



The Iterative Refinement Loop.



Live Exercise – Time to Practice.

‘Transform this weak prompt into a strong COSTAR prompt’



WEAK PROMPT

Create a blog post for my new
high-end coffee maker,
BrewMaster Pro.

YOUR IMPROVED COSTAR PROMPT

[Enter your improved prompt
here...]

Common Prompting Mistakes to Avoid.

- **Vague Instructions:** Providing overly broad commands that confuse the model. *Example:* 'Write a story.'
- **Lack of Context:** Forgetting to define the scenario or audience. *Example:* Ignoring that it's for 10-year-olds.
- **Assuming Persona Knowledge:** Omitting specific roles when requested. *Example:* Not specifying a 'medical context' for a health-related query.
- **Ignoring Length Constraints:** Letting output run on too long or too short. *Example:* Requesting a 'short summary' without setting a word count.
- **One-and-Done Thinking:** Expecting a perfect answer on the first attempt. *Example:* Not iterating through the 'Analyze & Improve' steps (referencing <IMAGE0> loop).

Session 1 – Key Takeaways

- 1 Define the Persona:** Explicitly state the role and expertise level for the AI.
- 2 Establish Clear Objectives:** Specify exactly what you want to achieve with the prompt.
- 3 Provide Critical Context:** Detail the scenario, audience, and relevance of the request.
- 4 Use Structured Frameworks:** Implement tools like COSTAR (Context, Objective, Style, Tone, Audience, Response) for consistency.
- 5 Embrace Iteration:** Don't expect perfection; plan for an 'Analyze & Improve' loop (referencing <IMAGE-1> concept).
- 6 Set Explicit Constraints:** Define length, format, and what to avoid (e.g., specific forbidden words).